NESTACK

Secure Multi-Tenant Records API

Technical Security Assessment Report

14 / 14

Automated Tests Passed

Final test execution: 14 passed, 1 warning

CANDIDATE DETAILS

Candidate	Venkata Naga Sahithya Mitta
Email	msahithya33@gmail.com
Assessment	Cybersecurity Intern / Engineer
Project	Secure Records API
Language	Python
Framework	FastAPI
Database	SQLite / SQLAlchemy
Testing	Pytest

Contents

1 Executive Summary **2**

2 Project Objectives **2**

3 Technology Stack **2**

4 Application Architecture **3**

4.1 Module Responsibilities 3

5 Security Implementation **3**

5.1 Password Security 3

5.2 Session Management 4

5.3 Authentication and Authorization 4

5.4 Multi-Tenant Access Control 4

5.5 Role Escalation Prevention 4

5.6 Brute-Force Protection 4

5.7 Input Validation 4

5.8 Request-Size Restriction 4

6 API Endpoint Reference **5**

7 Automated Security Testing **5**

8 Terminal Evidence **5**

8.1 Evidence 1: Automated Test Execution 5

8.2 Evidence 2: Git Repository Status 6

9 Threat Model **6**

10 Limitations and Future Improvements **6**

11 Conclusion **7**

1 Executive Summary

This project implements a secure multi-tenant Records API designed to protect user records from unauthorized access.

The application provides user registration, authentication, server-side session management, record creation and record retrieval.

Authorization is enforced at the individual record level. Standard users can access only their own records, while administrators are permitted to access records across users.

The implementation addresses important API security risks, including IDOR/BOLA, privilege escalation, insecure password storage, brute-force login attempts, invalid input and oversized HTTP requests.

Final Verification Result
14 automated tests passed successfully with zero failures.

2 Project Objectives

The main objectives of this assessment were:

- Implement secure user registration and login.
- Store passwords using Argon2id password hashing.
- Implement opaque server-side session authentication.
- Enforce ownership-based record authorization.
- Prevent unauthorized cross-user record access.
- Restrict administrator privileges.
- Prevent users from assigning themselves privileged roles.
- Implement account lockout after repeated failed logins.
- Validate incoming data and restrict oversized requests.
- Verify security controls using automated tests.

3 Technology Stack

Component	Technology
Programming Language	Python
API Framework	FastAPI
Database	SQLite
ORM	SQLAlchemy
Password Hashing	Argon2id
Session Token Generation	Python secrets module
Token Hashing	SHA-256
Request Validation	Pydantic
Authentication	HTTP Bearer
Testing Framework	Pytest

4 Application Architecture

The application follows a modular architecture that separates API routes, security logic, database operations, configuration and testing.

```
Nestack_Submission/
|
+-- app/
|   +-- main.py
|   +-- config.py
|   +-- database.py
|   +-- models.py
|   +-- schemas.py
|   +-- security/
|   |   +-- passwords.py
|   |   +-- sessions.py
|   |   +-- permissions.py
|   +-- routes/
|       +-- auth.py
|       +-- records.py
|
+-- tests/
|   +-- conftest.py
|   +-- test_auth.py
|   +-- test_access_control.py
|   +-- test_lockout.py
|
+-- scripts/
|   +-- create_admin.py
|
+-- README.md
+-- requirements.txt
```

4.1 Module Responsibilities

- `main.py`: Application initialization and request-size middleware.
- `models.py`: Database models and relationships.
- `schemas.py`: Request data validation.
- `passwords.py`: Password hashing and verification.
- `sessions.py`: Session creation, validation and revocation.
- `permissions.py`: Authentication and authorization.
- `auth.py`: Registration, login and logout.
- `records.py`: Record creation and retrieval.

5 Security Implementation

5.1 Password Security

Passwords are hashed using Argon2id before database storage. The application does not store user passwords in plaintext.

5.2 Session Management

The application implements opaque server-side sessions.

- Cryptographically secure tokens are generated using Python's `secrets` module.
- SHA-256 digests are stored instead of raw session tokens.
- Sessions have a configurable expiration period.
- Expired sessions are rejected.
- Logout revokes the corresponding server-side session.

5.3 Authentication and Authorization

Protected endpoints require a valid Bearer token. The server validates the token against the session database and identifies the authenticated user before granting access.

5.4 Multi-Tenant Access Control

Every record is associated with an owner through an `owner_id` field.

Normal users can retrieve only records belonging to their own account. Administrators can retrieve records across different users.

This ownership check mitigates IDOR/BOLA vulnerabilities.

5.5 Role Escalation Prevention

Public registration assigns the `user` role on the server side. Additional fields such as `role` are forbidden by request validation.

5.6 Brute-Force Protection

The login functionality tracks failed password attempts.

After five consecutive failed attempts, the account is temporarily locked for 15 minutes. Locked login attempts receive HTTP 429.

5.7 Input Validation

Username, password, record title and record body inputs are validated using request schemas.

Unexpected fields are rejected. Invalid request data is handled with HTTP 400.

5.8 Request-Size Restriction

A custom ASGI middleware enforces a maximum request body size of 16 KB. Oversized requests are rejected with HTTP 413.

6 API Endpoint Reference

Method	Endpoint	Purpose
GET	`/`	Application status
GET	`/health`	Health check
POST	`/register`	Register a user
POST	`/login`	Authenticate and issue a session
POST	`/logout`	Revoke the current session
POST	`/records`	Create a record
GET	`/records/{record_id}`	Retrieve an authorized record

7 Automated Security Testing

The following 14 tests were executed.

No.	Test Case	Result
1	Unauthenticated access is rejected	PASS
2	User can access own record	PASS
3	User cannot access another user's record	PASS
4	Administrator can access another user's record	PASS
5	Logout revokes session	PASS
6	Oversized request is rejected	PASS
7	Zero record ID returns HTTP 400	PASS
8	Non-integer record ID returns HTTP 400	PASS
9	User registration works	PASS
10	Duplicate registration is rejected	PASS
11	Successful login works	PASS
12	Invalid login is rejected	PASS
13	Role injection is blocked	PASS
14	Account lockout after five failed attempts works	PASS

8 Terminal Evidence

8.1 Evidence 1: Automated Test Execution

Command executed in the project virtual environment:

```
python -m pytest -v
```

Observed terminal output:

```
collected 14 items

tests/test_access_control.py::test_unauthenticated_access PASSED
tests/test_access_control.py::test_user_can_access_own_record PASSED
tests/test_access_control.py::test_user_cannot_access_another_users_record PASSED
tests/test_access_control.py::test_admin_can_access_another_users_record PASSED
tests/test_access_control.py::test_logout_revokes_session PASSED
tests/test_access_control.py::test_oversized_request_rejected PASSED
tests/test_access_control.py::test_zero_record_id_returns_400 PASSED
tests/test_access_control.py::test_non_integer_record_id_returns_400 PASSED
tests/test_auth.py::test_registration PASSED
tests/test_auth.py::test_duplicate_registration PASSED
tests/test_auth.py::test_successful_login PASSED
tests/test_auth.py::test_invalid_login PASSED
```

```
tests/test_auth.py::test_role_injection_blocked PASSED
tests/test_lockout.py::test_account_lockout_after_five_failed_attempts PASSED

14 passed, 1 warning in 5.98s
```

TEST RESULT: 14 PASSED / 0 FAILED

The test run reported one Starlette deprecation warning concerning the HTTPX integration with TestClient. It did not cause any test failures.

8.2 Evidence 2: Git Repository Status

Command executed:

```
git status
```

Observed terminal output:

```
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
```

This confirms that the local main branch was reported as synchronized with its configured remote branch and that there were no uncommitted changes at the time of the check.

This output does not independently verify repository visibility or collaborator permissions.

9 Threat Model

Threat	Implemented Mitigation
Password database disclosure	Argon2id password hashing
Session token database disclosure	Store token digests, not raw tokens
Cross-tenant record access	Ownership-based authorization
IDOR / BOLA	Owner ID verification
Privilege escalation	Server-controlled role assignment
Brute-force login attempts	Five-attempt temporary lockout
Session reuse after logout	Server-side session revocation
Malformed input	Request schema validation
Oversized requests	16 KB request-size restriction

10 Limitations and Future Improvements

This implementation is an assessment prototype and should not be considered production-hardened without further review.

Recommended improvements include:

- Use PostgreSQL or another production-grade database.
- Make failed-login counting and lockout updates atomic to handle concurrent login attempts.
- Add IP-level rate limiting and security audit logging.

- Add dedicated tests for session expiration and token digest persistence.
- Deploy behind HTTPS with secure operational configuration.
- Add audit trails for administrator actions.

These are recommendations and are not claimed as implemented or tested in the current assessment.

11 Conclusion

The Secure Multi-Tenant Records API demonstrates practical implementation of authentication, opaque server-side sessions, role-aware authorization, tenant isolation, password hashing, login lockout and request validation.

The supplied terminal evidence records a successful execution of all 14 automated tests and a clean Git working tree.

The project demonstrates an understanding of secure API development, common authorization vulnerabilities and application security testing.

END OF REPORT

Nestack Candidate Technical Assessment